

Virtual Memory

Ivy Peng

Associate Professor

EECS, KTH



Outline

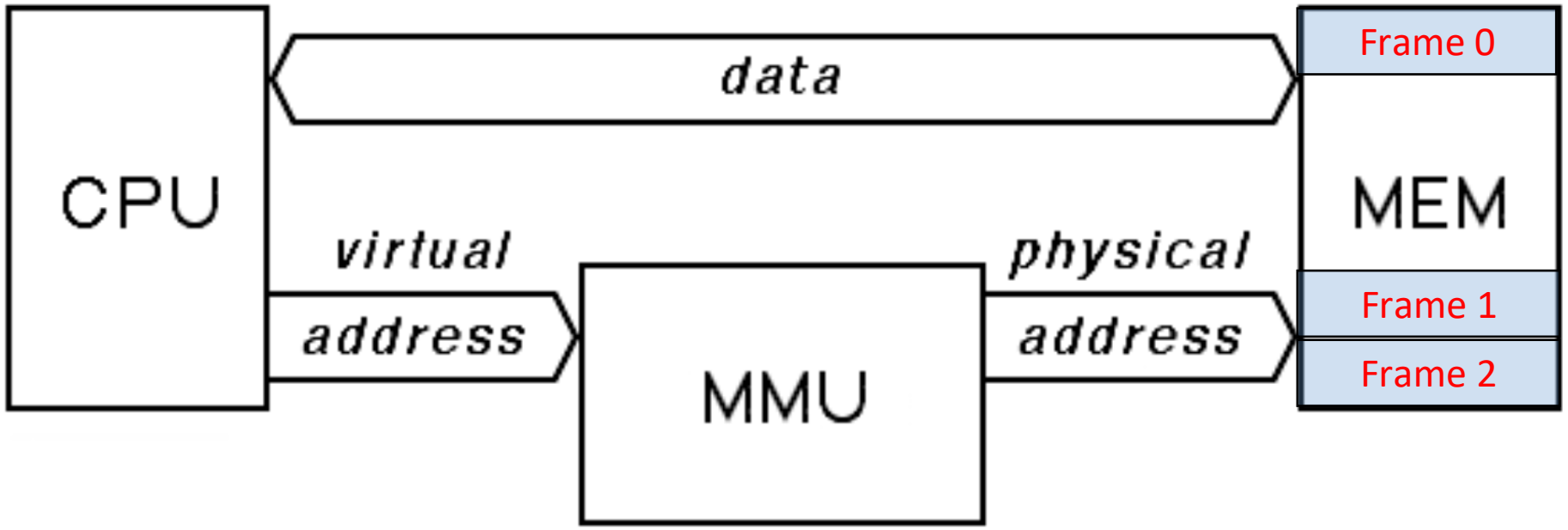
- **Paging**
- Page Tables:
  - Hierarchical Page Table
  - Hashed Page Table
  - Inverted Page Table
- VM system calls
- Linux's Buddy system and SLAB allocator

By the End of This Lecture

- Q1 How does paging translate a logical address to physical?
- Q2 Why a TLB, and how do you compute effective access time?
- Q3 Why hierarchical, hashed, and inverted page tables?

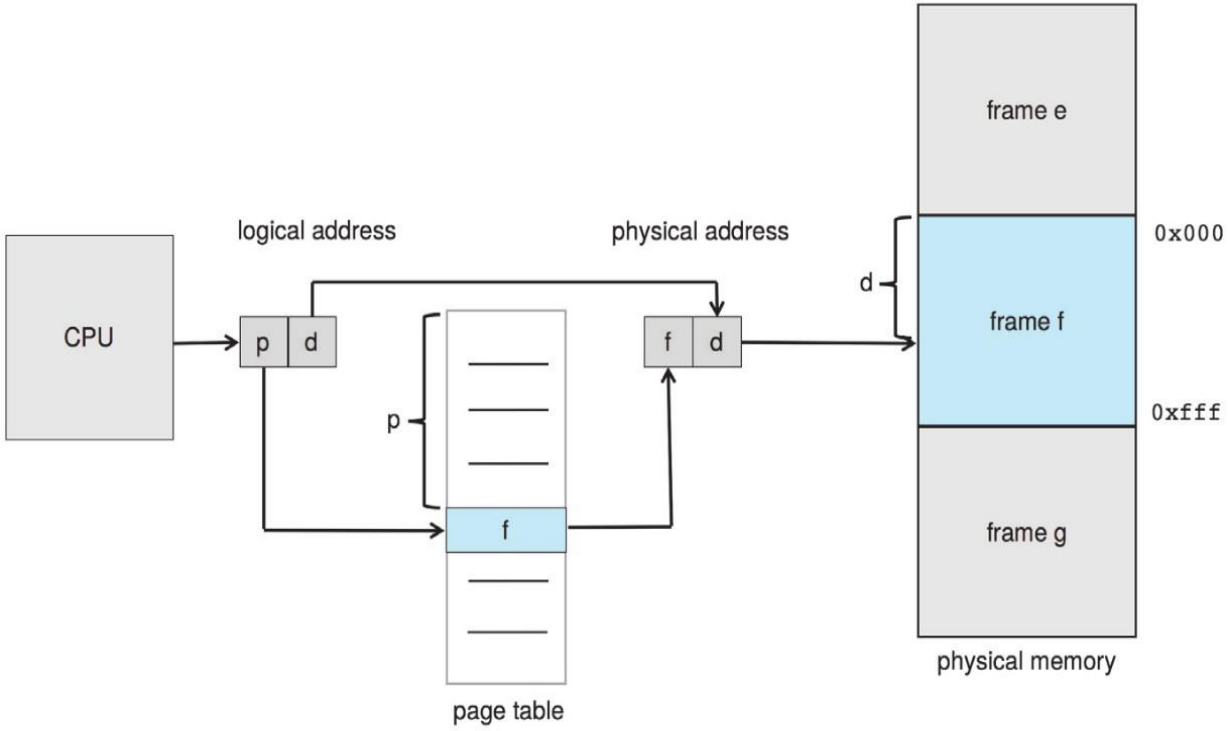
What is Paging?

- Give each process a large, contiguous virtual (logical) address space
  - Divide **logical/virtual address space** into fixed-sized blocks called **pages**
  - Divide **physical memory** into fixed-sized blocks called **frames**.
  - Each page can be backed by a frame scattered anywhere in RAM
- 4KB page size is most commonly



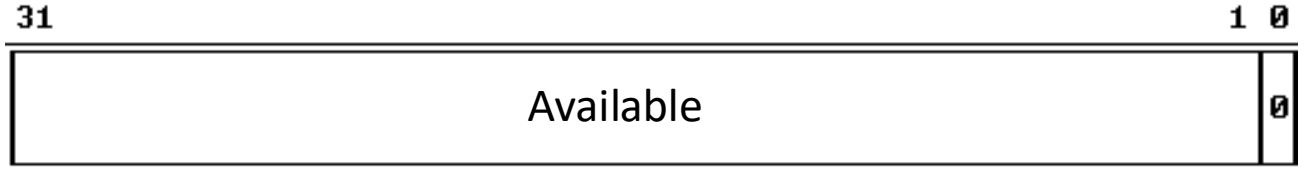
Page Table

- OS keeps track of all free frames in physical memory
- To run a program of size N pages, need to find N free frames and load the program.
- Set up a **page table** to translate/map logical to physical addresses.

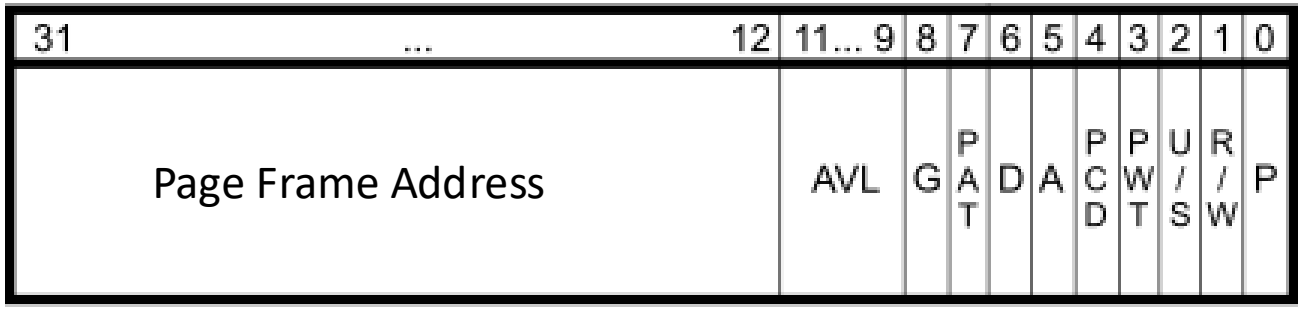


Page Table Entry (PTE)

- An example of Page Table entry for 32-bit virtual address. PTEs take space in memory, e.g., 4 bytes.
- If the page is invalid, i.e., not in physical memory yet. The Present bit is 0



- If the page is valid, i.e., already in physical memory yet



P: Present
R/W: Read/Write
U/S: User/Supervisor
PWT: Write-Through
PCD: Cache Disable
A: Accessed
D: Dirty
G: Global
AVL: Available
PAT: Page Attribute Table

Optional: Page and Page Table in Linux

/ include / linux / mm_types.h

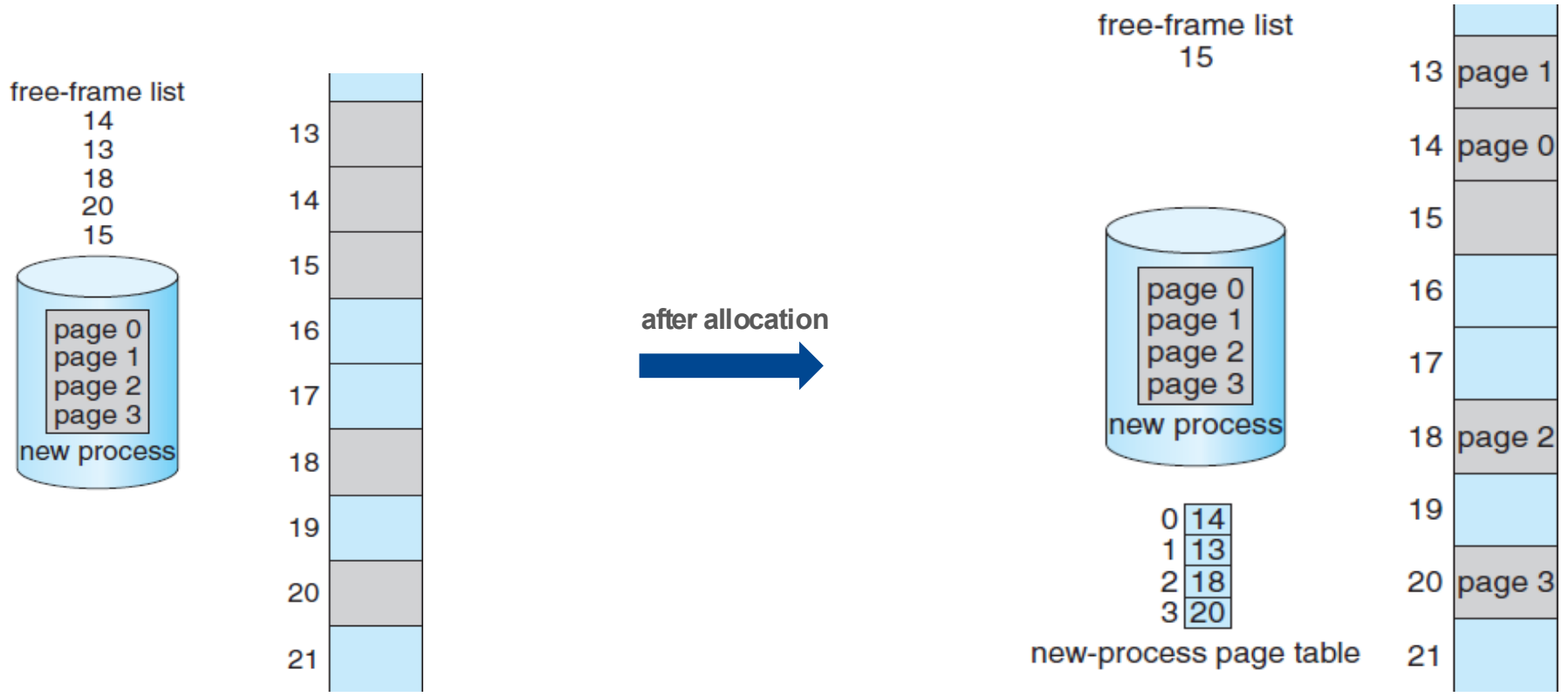
```
72  struct page {
73          unsigned long flags;            /* Atomic flags, some possibly
74                                           * updated asynchronously */
75          /*
76           * Five words (20/40 bytes) are available in this union.
77           * WARNING: bit 0 of the first word is used for PageTail(). That
78           * means the other users of this union MUST NOT use the bit to
79           * avoid collision and false-positive PageTail().
80           */
81          union {
82                  struct {        /* Page cache and anonymous pages */
83                          /**
84                           * @lru: Pageout list, eg. active_list protected by
85                           * lruvec->lru_lock.  Sometimes used as a generic list
86                           * by the page owner.
87                           */
88                          union {
89                                  struct list_head lru;
90
91                                  /* Or, for the Unevictable "LRU list" slot */
92                                  struct {
93                                          /* Always even, to negate PageTail */
94                                          void *__filler;
95                                          /* Count page's or folio's mlocks */
96                                          unsigned int mlock_count;
97                                  };
98
99                                  /* Or, free page */
100                                 struct list_head buddy_list;
101                                 struct list_head pcp_list;
102                         };
103                         /* See page-flags.h for PAGE_MAPPING_FLAGS */
104                         struct address_space *mapping;
105                         union {
106                                 pgoff_t index;          /* Our offset within mapping. */
107                                 unsigned long share;    /* share count for fsdax */
108                         };
109                         /**
110                          * @private: Mapping-private opaque data.
111                          * Usually used for buffer_heads if PagePrivate.
112                          * Used for swp_entry_t if swapcache flag set.
113                          * Indicates order in the buddy system if PageBuddy.
114                          */
115                         unsigned long private;
116                 };
117                 struct {        /* page_pool used by netstack */
118                         /**
119                          * @pp_magic: magic value to avoid recycling non
120                          * page_pool allocated pages.
121                          */
```

/ include / linux / mm_types.h

```
820 struct mm_struct {
821         struct {
822                 /*
823                  * Fields which are often written to are placed in a separate
824                  * cache line.
825                  */
826                 struct {
827                         /**
828                          * @mm_count: The number of references to &struct
829                          * mm_struct (@mm_users count as 1).
830                          *
831                          * Use mmgrab()/mmdrop() to modify. When this drops to
832                          * 0, the &struct mm_struct is freed.
833                          */
834                         atomic_t mm_count;
835                 } ____cacheline_aligned_in_smp;
836
837                 struct maple_tree mm_mt;
838
839                 unsigned long mmap_base;        /* base of mmap area */
840                 unsigned long mmap_legacy_base; /* base of mmap area in bottom-up allocatio
841 #ifdef CONFIG_HAVE_ARCH_COMPAT_MMAP_BASES
842                 /* Base addresses for compatible mmap() */
843                 unsigned long mmap_compat_base;
844                 unsigned long mmap_compat_legacy_base;
845 #endif
846                 unsigned long task_size;        /* size of task vm space */
847                 pgd_t * pgd;
848
849 #ifdef CONFIG_MEMBARRIER
850                 /**
851                  * @membarrier_state: Flags controlling membarrier behavior.
852                  *
853                  * This field is close to @pgd to hopefully fit in the same
854                  * cache-line, which needs to be touched by switch_mm().
855                  */
856                 atomic_t membarrier_state;
857 #endif
858
859                 /**
860                  * @mm_users: The number of users including userspace.
861                  *
862                  * Use mmget()/mmget_not_zero()/mmput() to modify. When this
863                  * drops to 0 (i.e. when the task exits and there are no other
864                  * temporary reference holders), we also release a reference on
865                  * @mm_count (which may then free the &struct mm_struct if
866                  * @mm_count also drops to 0).
867                  */
868                 atomic_t mm_users;
869
```

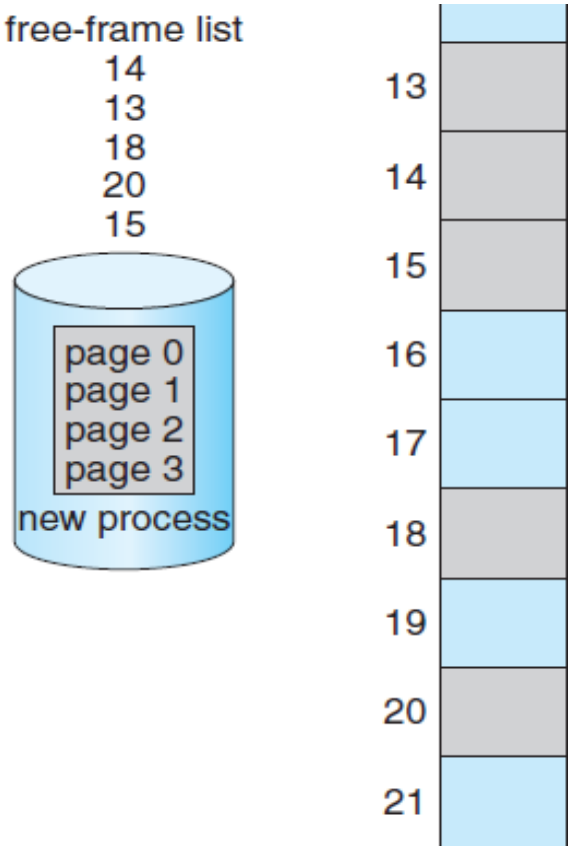
Paging: Frame Allocation

- The process gets a contiguous virtual address space, but pages (**frames**) in the physical memory are scattered



Paging: Fragmentation

- Paging avoids external fragmentation and the need for compaction
- But internal fragmentation remains



- Page size = 2048 bytes, Process size = 72766 bytes.
- What is the fragmentation ratio here?
  - Fragmentation Ratio = (Allocated – Requested)/Allocated
  - Requested = 72766
  - Allocated = #pages * page_size
  - ~1.3%

Paging: Page Size

- On average fragmentation ≈ o.5 page size
  - Thus, small page sizes are desirable for reducing fragmentation.
- Small pages need more space for maintaining page-table
  - Assume the size of each page table entry (PTE) is 4 bytes
  - The number of PTE increases when page size is smaller
  - Large server class machines can have 1TB memory, calculate the overhead for table?
    - 2^40 / 2^12 * 4B= 2^28 PTE * 4B = 2^30 B = 1 GB
- Page Sizes can be system specific, 4KB is common on x86, 64KB on ARM servers.

The system call *getpagesize()*

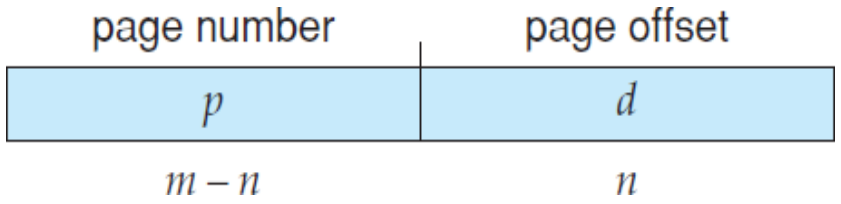
Or

Command line: *getconf PAGESIZE*

Paging: Address Translation

- Logical address generated by CPU on a m-bit system has **m-bits**, e.g., Intel 32-bit or ARM 64-bit systems like ARMv8-64 architecture
- The **m-bit** virtual address is divided into two parts:
  - Page number (**p**): used as an index into a page table that contains base address of each page in physical memory.
  - Page offset (**d**): combined with base address to define the physical memory address.

For given logical address space on a m-bit system 2^m and page size 2^n.

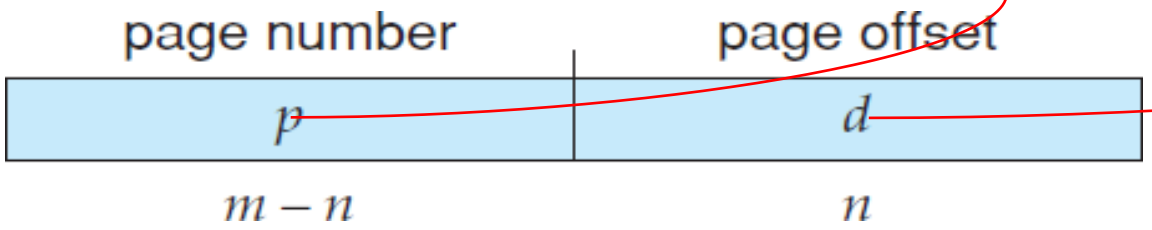


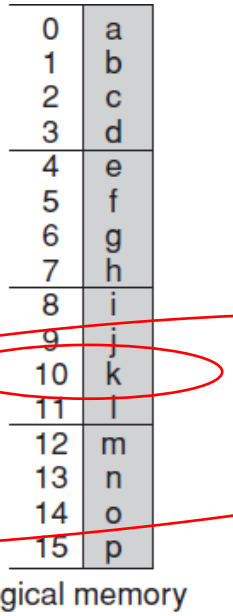
For a page size of 4KB, what is the value of d?

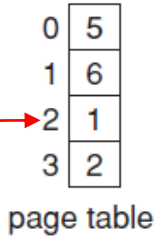
Address Translation

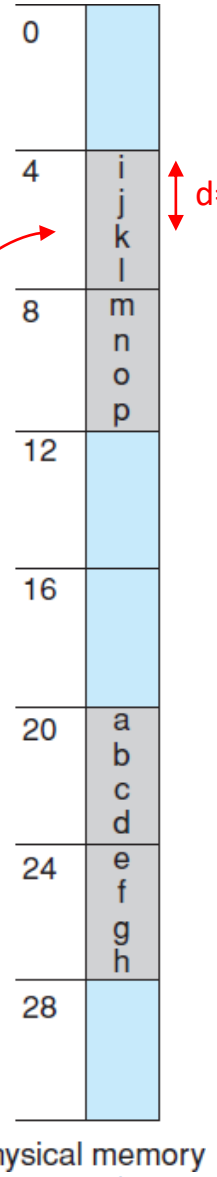
if page size is 4:

- What is the physical address for logical address 0xA?
- What are the values loaded from address OxA?







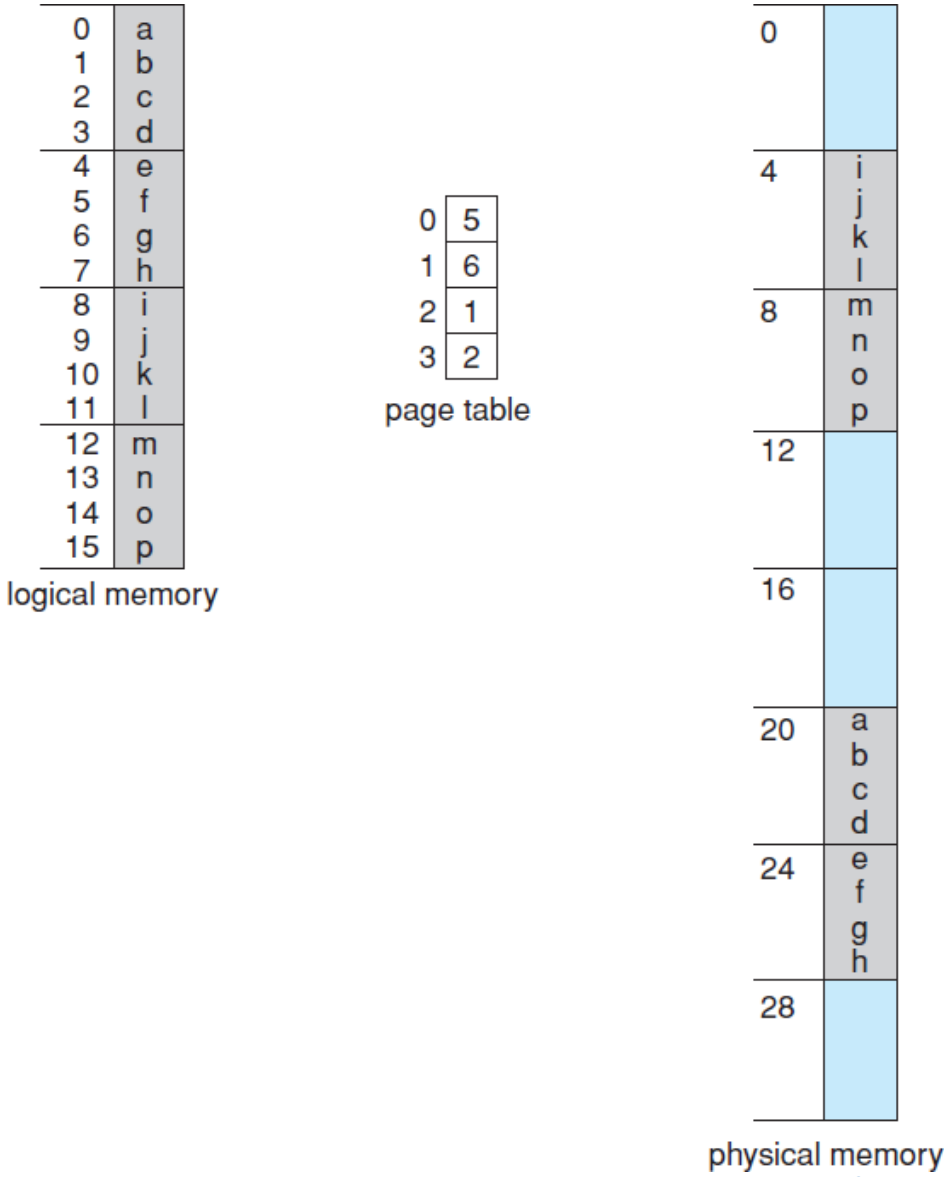


- Page Size = 4 → n = 2
- Offset = 0xA mod 4 = 2
- Page number = (0xA / 4) = 2 → page frame number (PFN) = 1
- Physical address: PFN * page size + offset = 1*4+2 = 6
- Data values?

Address Translation

- Given the logical address: m = 4 and n = 2
  - How many bit in a virtual address?
  - What is the page size?

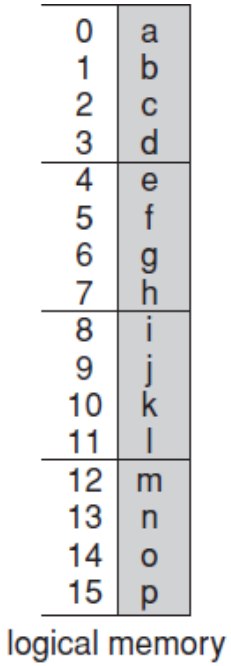
- What is the physical address for logical address X?
  - First, determine p and d
    - p = X/page_size
    - d = X%page_size
  - Next, look at the page table to find the frame of p

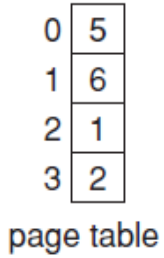


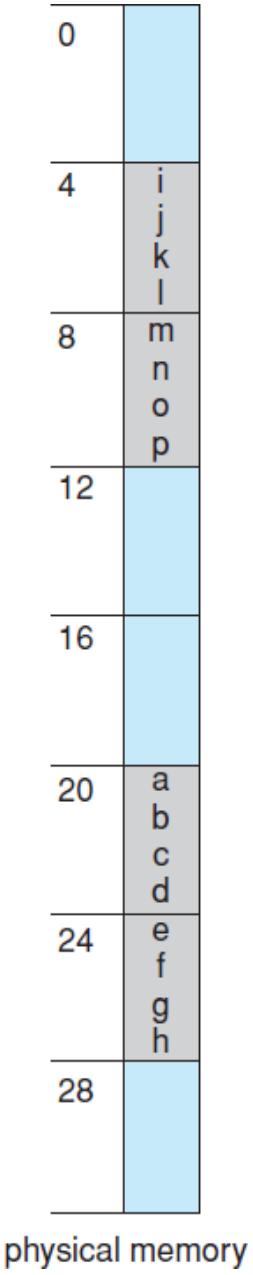
More Examples

- What is the physical address for Logical address 3?

- What is the value loaded for Logical address 10?





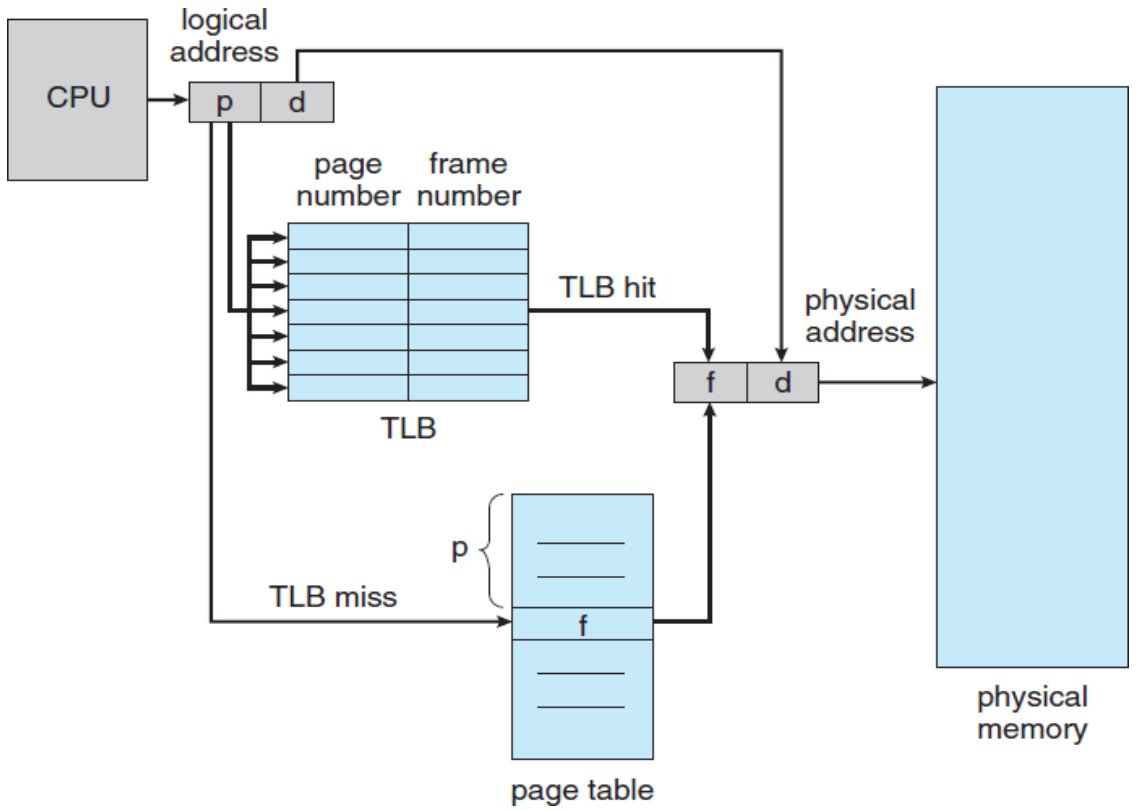


Translation Look-aside Buffers (TLB)

- Page table is kept in main memory.
- How many memory accesses are required for each data/instruction access?

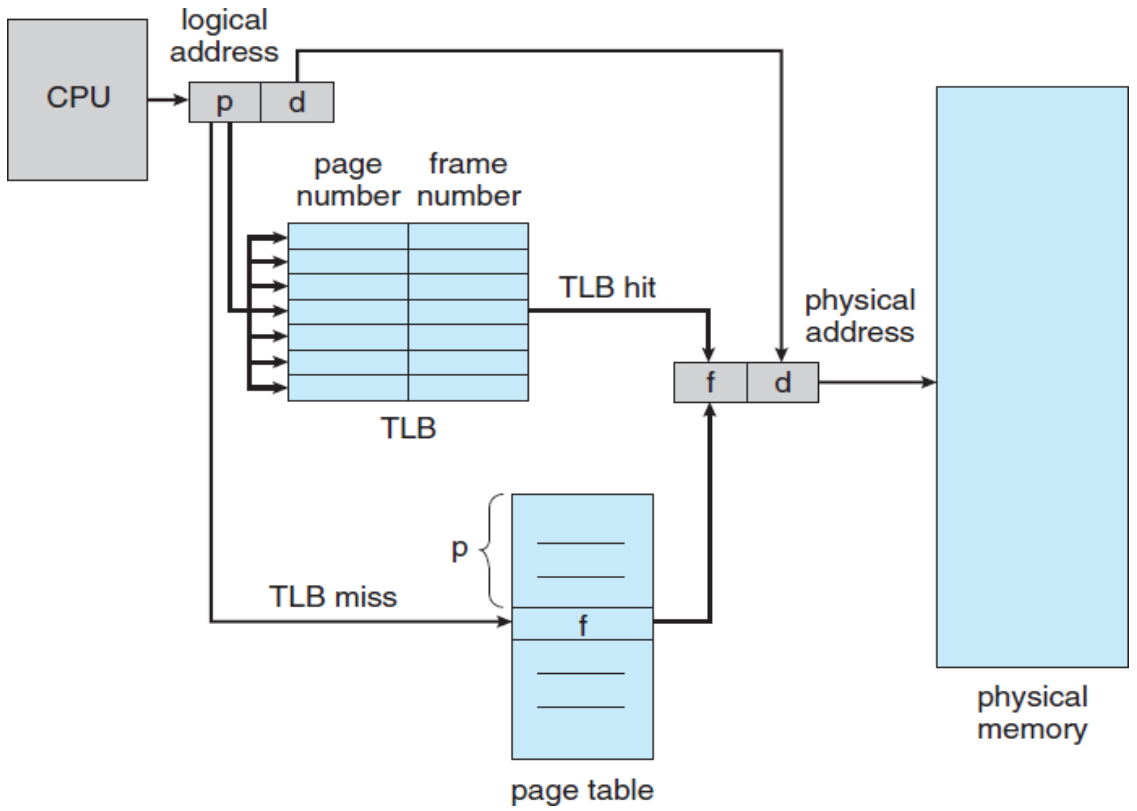
- Access main memory is slow
- TLB is a special fast-lookup hardware cache for translating addresses

- Address translation (p, d) is now similar to data cache lookup



Translation Look-aside Buffers (TLB)

- Address translation (p, d) is now similar to data cache lookup:
  - If p is in TLB, i.e., is cached already, get frame# out from TLB, no need to access main memory.
  - Otherwise, get frame# from page table in main memory



Effective Access Time with (1-level) TLB

- Typical range of latency to TLB
  - Hit latency: 0.5 - 1 clock cycle, e.g., 10 nanoseconds
  - Miss latency: 10 - 100 clock cycles, e.g., 100 nanoseconds

- Typical hit ratio in TLB is quite high, say 99%

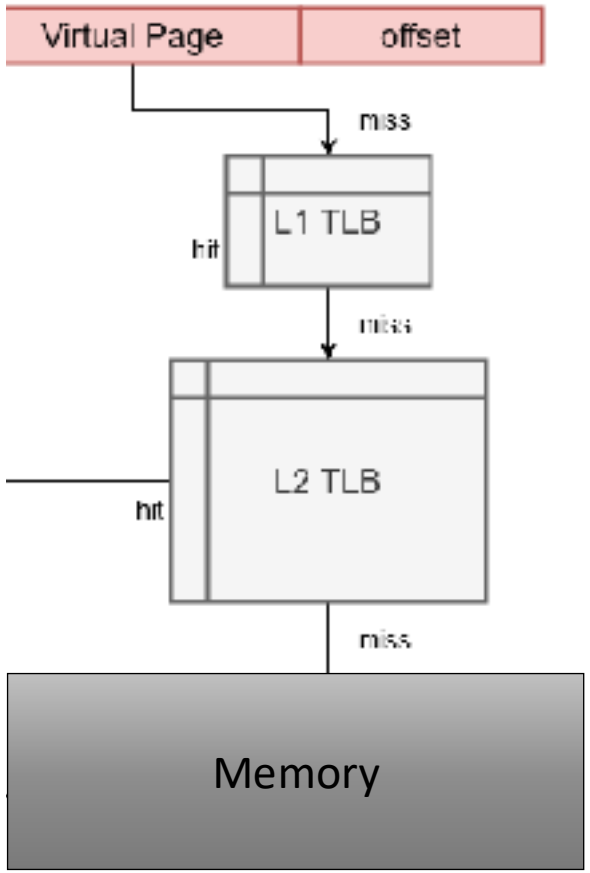
- **Effective Access Time** = hit ratio * hit latency + miss ratio * **miss latency** (= TLB L1 lookup + main memory access)
- = 99% * 10 + 1% * (10 + 100)
- = 11 nanoseconds

How about 2-level and 3-level TLB?

Effective Access Time with (multi-level) TLB

If 2-level TLB, what is Effective Access Time?

- L1 Hit/lookup latency: e.g., 10 nanoseconds;
- L2 Hit/lookup latency: e.g., 30 nanoseconds;
- Memory latency: e.g., 100 nanoseconds
- Suppose TLB hit ratios in level 1 and level 2 are 90% and 80%, respectively
  - 90% * 10 -------> hit in L1
  - + (1-90%)*80%*(10+30) -------> hit in L2
  - + ((1-90%)*(1-80%)) * (10+30+100) -------> miss in L2



- Now extrapolate to N-level
  - **miss latency in level N** = TLB lookup latency in all (N-1) levels
  - + main memory access latency



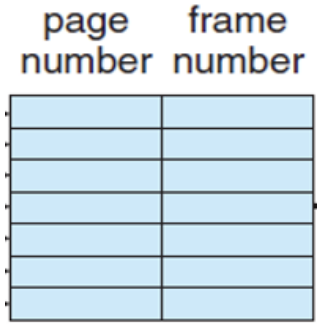
Outline

- Paging
- **Page Tables:**
  - **Hierarchical Page Table**
  - **Hashed Page Table**
  - **Inverted Page Table**
- VM system calls
- Linux's Buddy system and SLAB allocator

Structure of Page Tables

- So far, we have only considered 1-level page table

- Consider a 32-bit logical address space and Page Size of 4KB
  - m = 32
  - n = 12
- Page table would have 1 million entries
  - (m – n) = 20 bits left, giving us 2^{20}
- If each entry in the page table takes 4Bytes:
  - 4MB of physical address space memory is used for page table alone.



1-level

- Now consider 64-bit logical address space, which is more common nowadays
  - For page size 64KB, how many bytes would the page table take?
    - 4Bytes per entry * # entry (=2^(64-16)) = 2^49 bytes
    - Unrealistic!

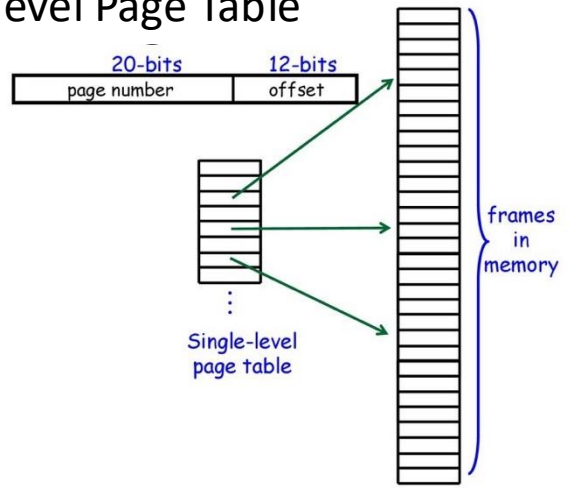
Hierarchical Page Tables

- The amount of memory used for storing Page Tables can cost a lot.
- Don’t want to allocate that contiguously in main memory.

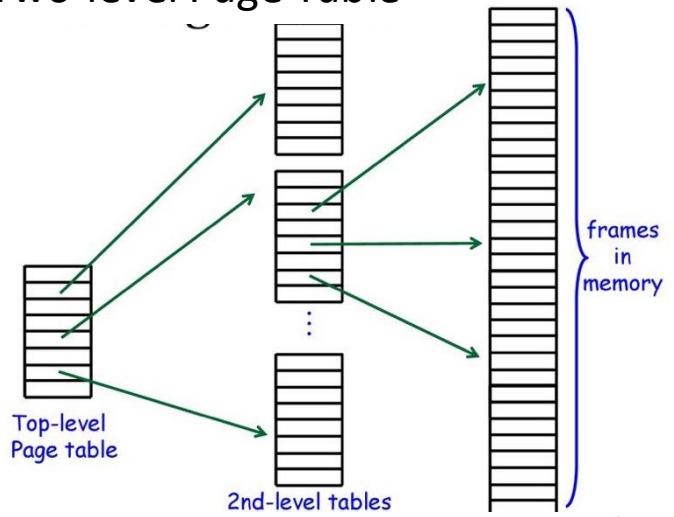
- Hierarchical paging breaks up the logical address space into multiple page tables, then we page the page tables
  - Each page table is one page (frame)
  - The addressable size is determined by how many entries fit in a page table

- A simple example is a two-level page table.

One-level Page Table



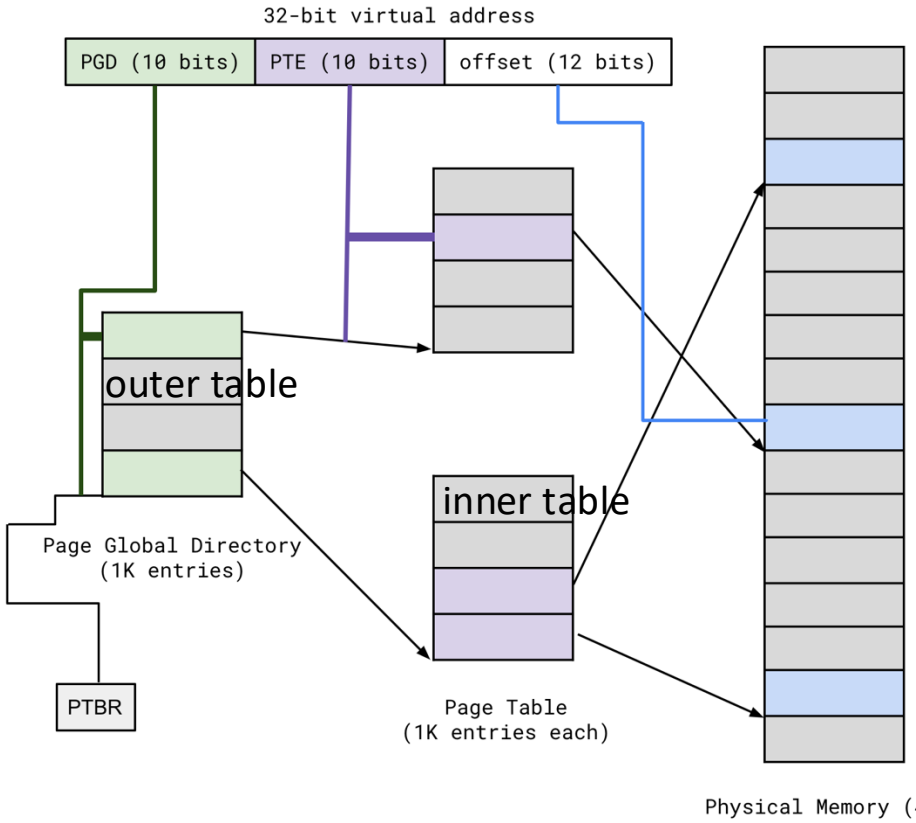
Two-level Page Table



Hierarchical Paging – Two-level

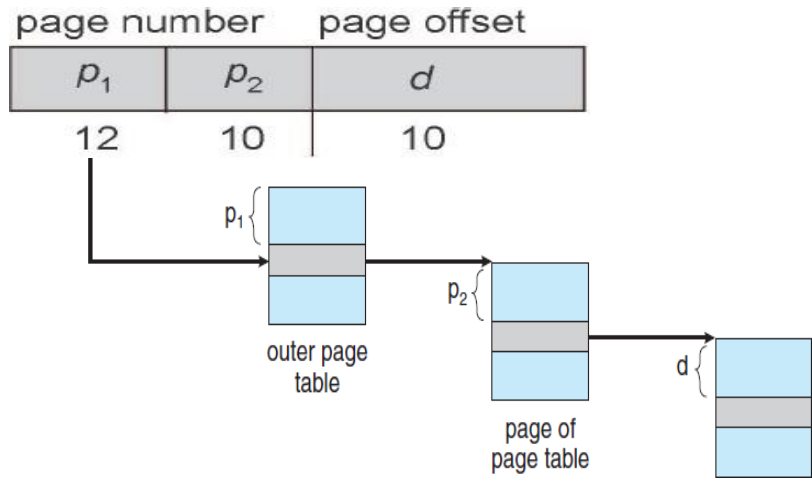
- The logical address (e.g., 32-bit) is now split into 3 parts. A PTE takes 4-byte.
- How much memory address can it address?

1K inner tables x 1K pages per inner tables x 4KB per page = 4GB



Another Example

- A PTE takes 4-byte. On a 32-bit machine, address is divided:
  - A page number consisting of 22 bits.
  - A page offset consisting of 10 bits.
- The page number is further divided into:
  - A 12-bit page number is an index into the outer page table
  - A 10-bit page offset is the displacement within the page of the inner page table



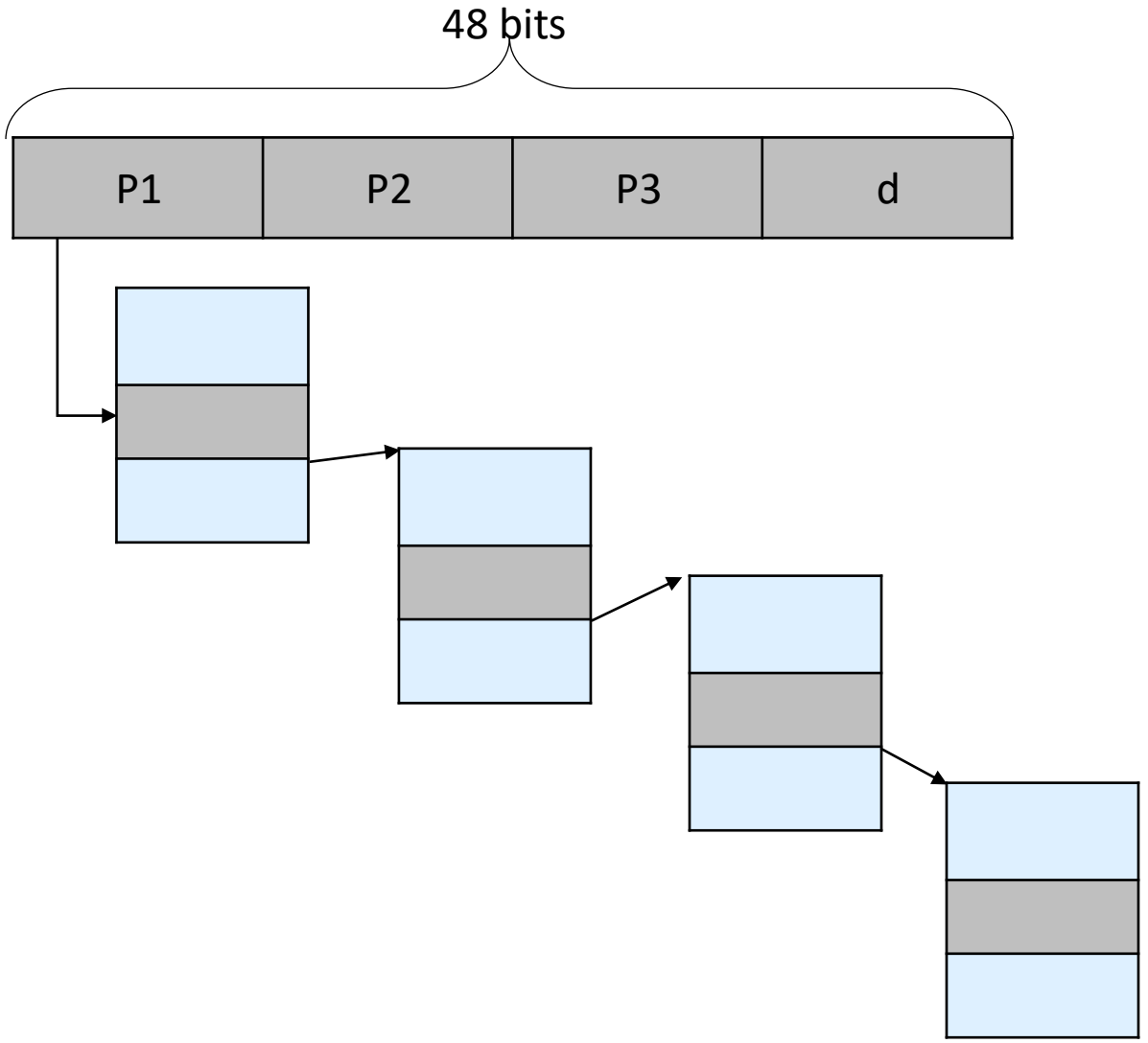
- How much memory can be addressed?

256 inner tables x 256 pages per inner tables x 1KB per page = 64MB

Three-level Paging Example

- Assume a page size of 128KB -> d = 17
- Assume 48-bit virtual address is used and each page table entry is 4B

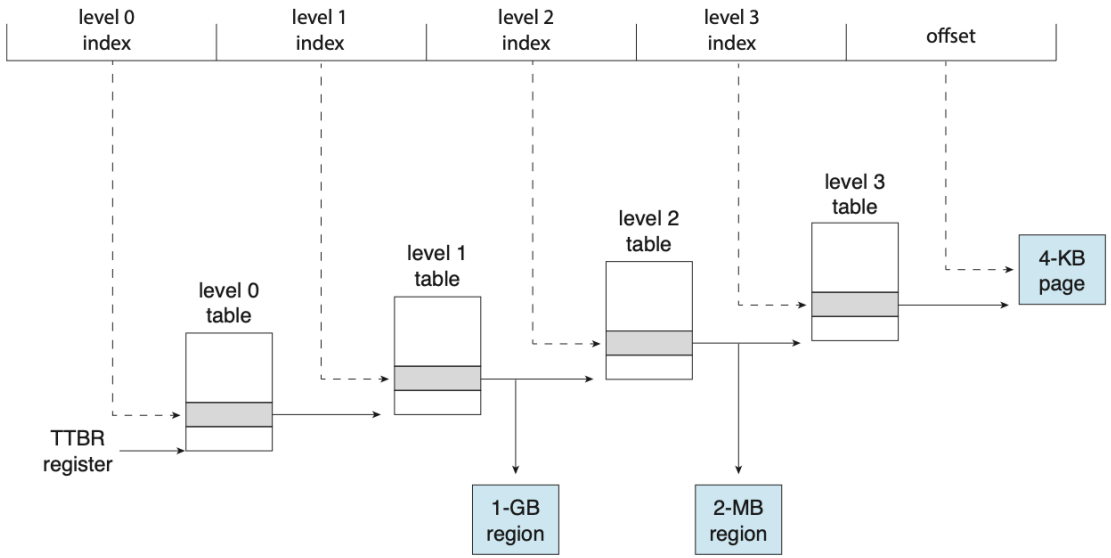
- If we want every page table exactly fits into a single page, how many bits do P1, P2, P3 have?
  - We need to address $2^{48} / 2^{17} = 2^{31}$ pages in total
  - Each page table has 128KB/4B = 2^{15} entries
  - Start from P3 = 15, P2 = 15, P1 = 1



Hierarchical Paging for 64-bit machines

- In practice, x86-64 architecture currently provides a 48-bit virtual address with support for different page sizes
  - 4 KB, 2 MB, or 1 GB using four levels of paging hierarchy.

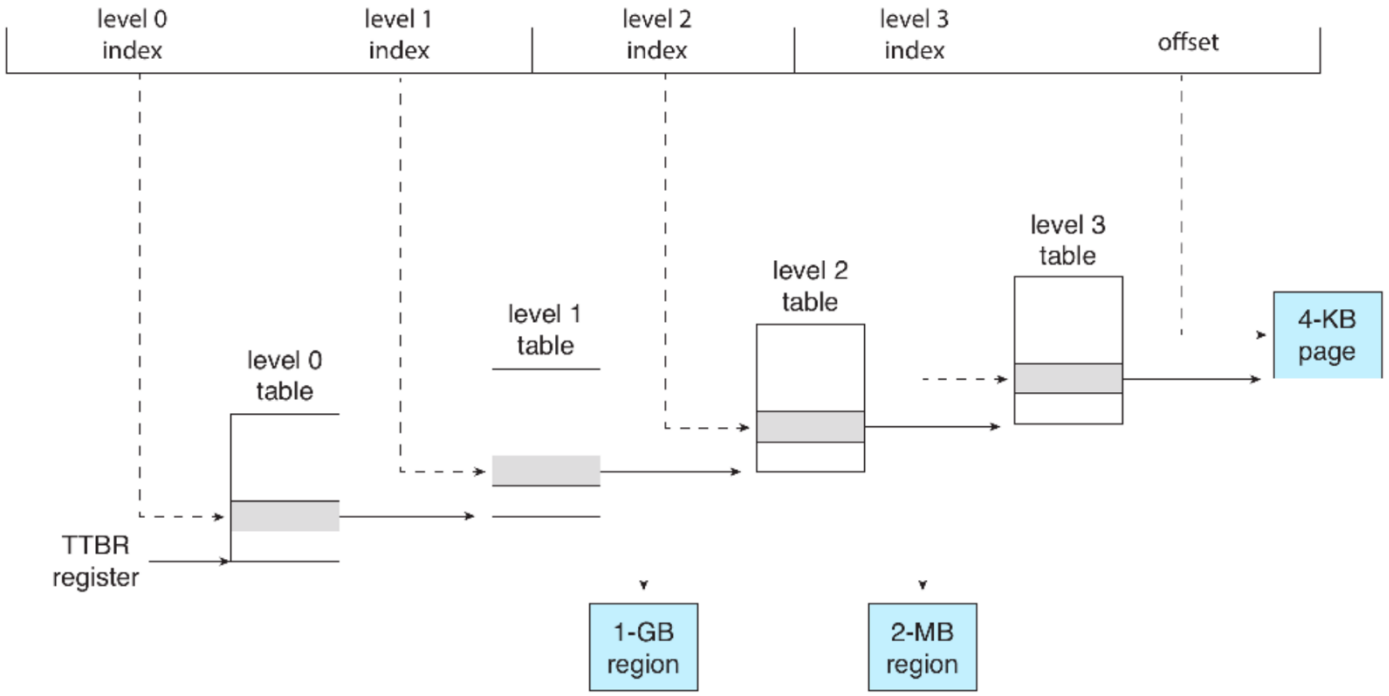
- ARMv8-64 architecture, multi-level page tables
  - the table entries for level 1 and level 2 may use 1GB or 2MB page sizes
  - the table entries for level 4 still refers to 4KB pages



ARMv8 4-level paging

Case Study: ARMv8

- 4-level hierarchical paging for a 4 KiB Page
  - Page offset: 12 bit
  - Page number: 4·9 bit
  - Table sizes: 2^9 entries, 64 bit per page entry $\Rightarrow 2^{12}$ bit = 4 KiB



Case Study: ARMv8

- Default maximum logical address width: 48 bit
  - With Large Virtual Address support: 52 bit

- Support of 1 or 2 virtual address ranges
  - AArch64 Linux memory layout:
    - Lower range for use by user
    - Upper range for use by kernel

- Memory region: Range of virtual addresses
  - Memory region attributes can be set in hardware
  - Example attributes: memory type, cacheability, shareability

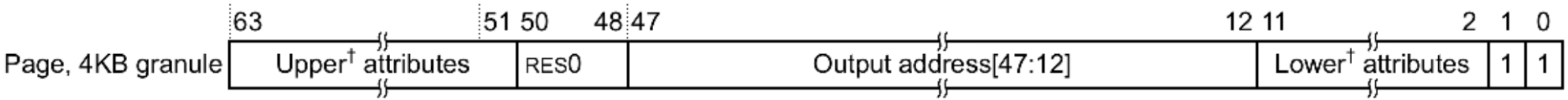
- Supports three different translation granularities

Granularity Size	Page Size	Region Size
4KiB	4KiB	2 MiB, 1GiB
16KiB	16KiB	32 MiB
64KiB	64KiB	512 MiB

Case Study: ARMv8

ARMv8 **TLB Entry Content**

- Selected upper attributes
  - XN: Execute never bit
  - PXN: Privileged execute never bit
- Selected lower attributes
  - AF: Access flag (0 if page has never been accessed)
  - AP: Data access permission bits
  - nG: non-global bit decides on whether entry is global or process-specific



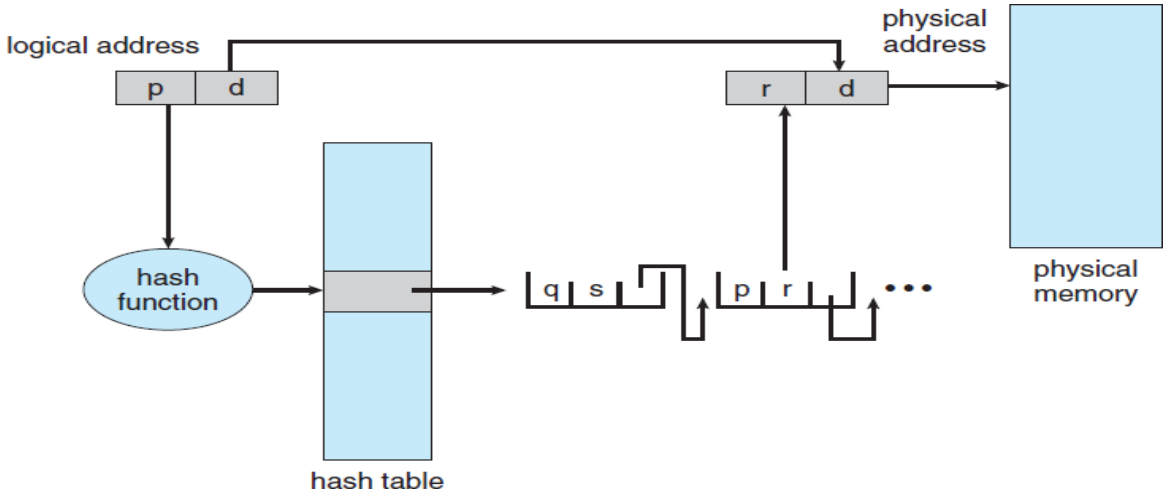


Outline

- Paging
- Page Tables:
  - Hierarchical Page Table
  - **Hashed Page Table**
  - Inverted Page Table
- VM system calls
- Linux's Buddy system and SLAB allocator

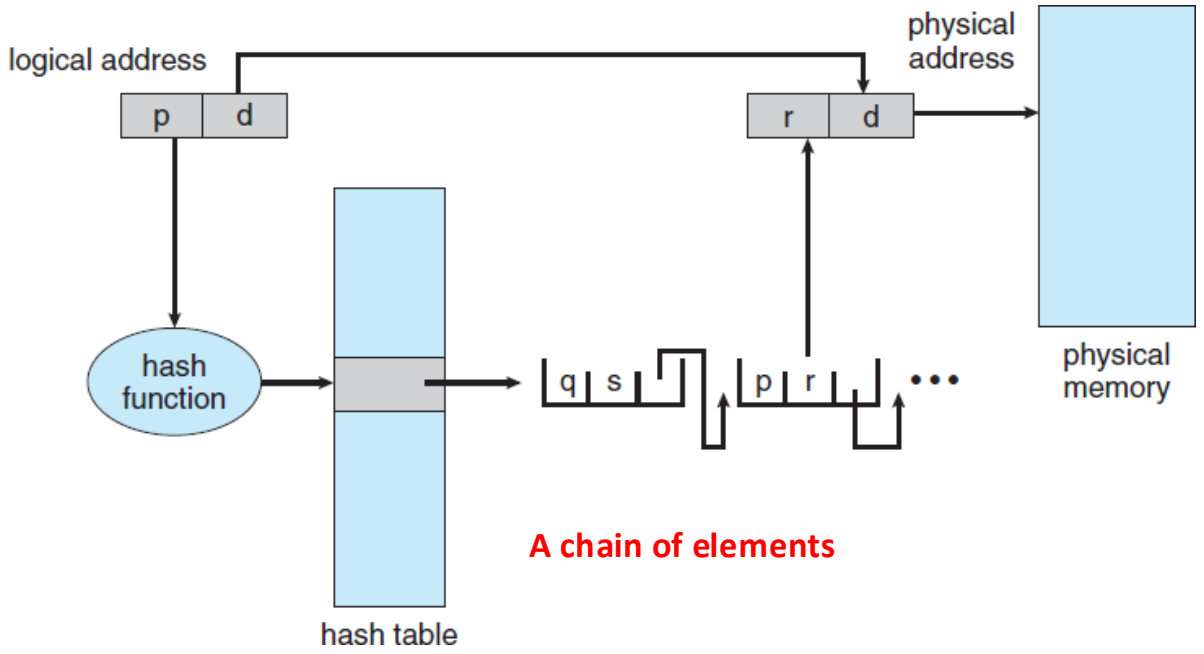
Hashed Page Tables

- For address space larger than 32-bit, hierarchical paging could be inefficient:
  - **Page Walk** requires multiple memory accesses to multi-level page tables
  - Multiple level TLBs
- Instead, hashed page table can be used
  - the virtual page number is the hash value
  - Each entry in the hash table contains a linked list of elements
  - Each element consists of (1) the virtual page number, (2) page frame, and (3) a pointer to the next element in the linked list



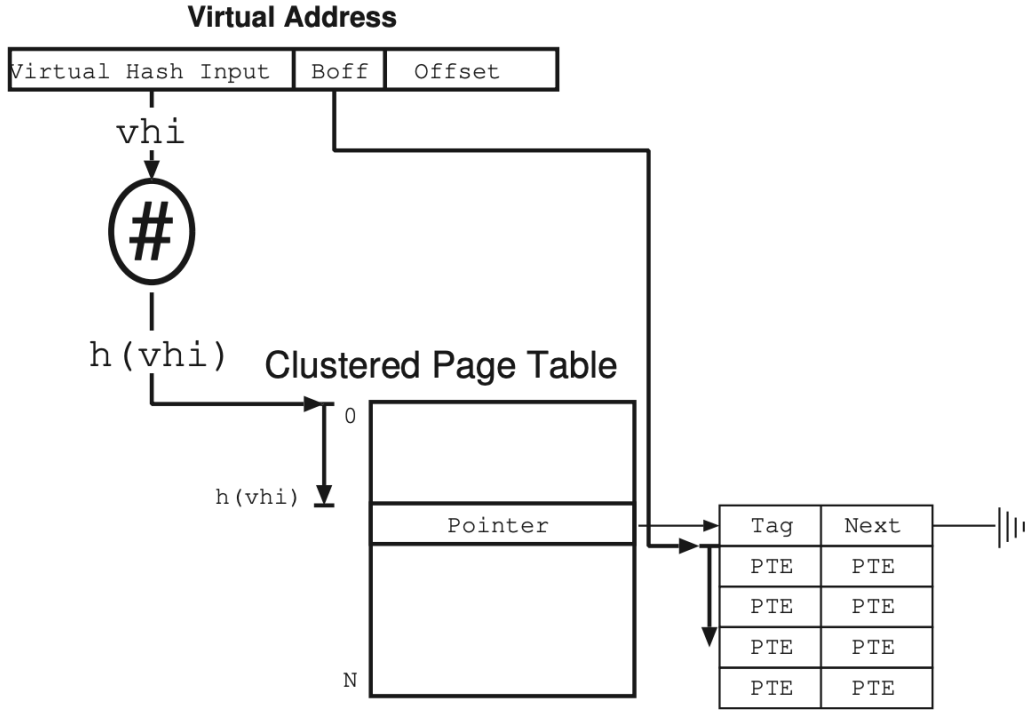
Hashed Page Tables

- The logical page number is hashed into a page table.
- This page table contains a chain of elements that hashed to the same location (collision)
- Each element contains
  - The logical page number
  - The value of the mapped page frame
  - A pointer to the next element



Clustered Page Tables

- It is a modification of hashed page tables
- Each entry in the table contains **a cluster of PTEs for contiguous pages**
- Suitable for sparse address spaces: memory references are noncontiguous and scattered

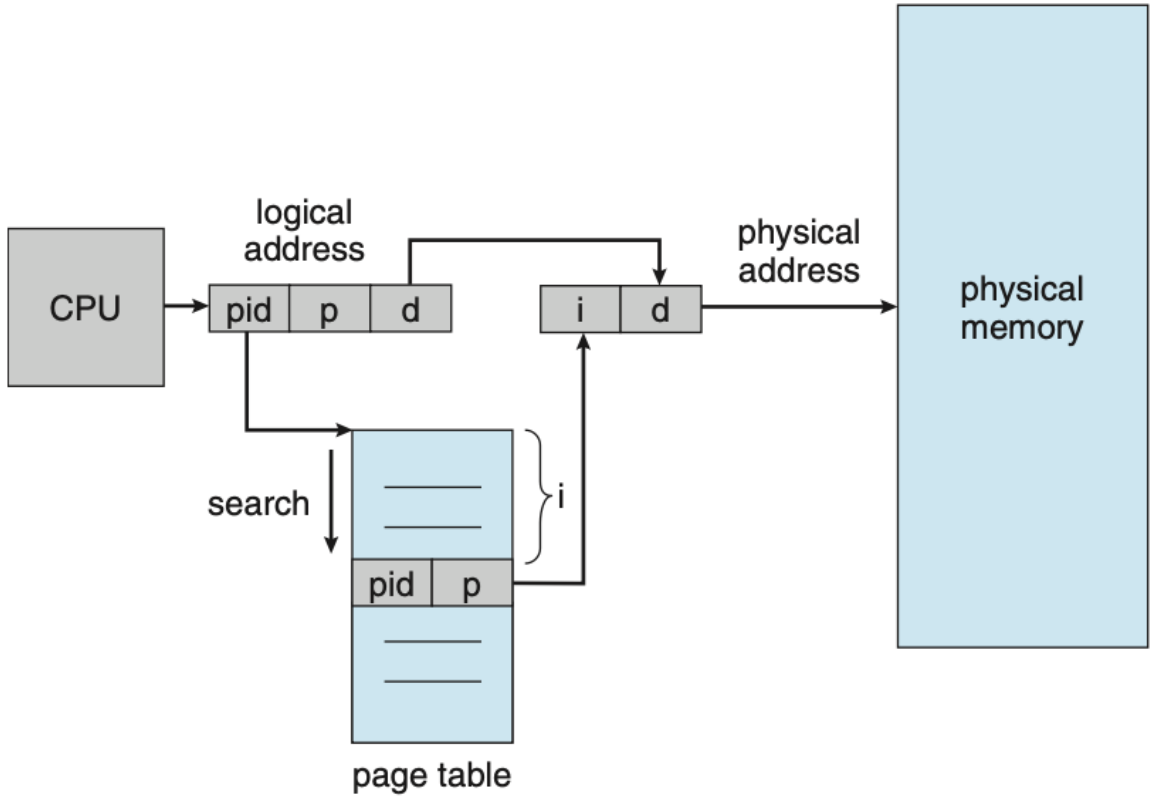


Talluri, M., Hill, M.D. and Khalidi, Y.A., 1995, December. A new page table for 64-bit address spaces. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles* (pp. 184-200).

Inverted Page Tables

- So far, hierarchical page tables and hashed page tables are created per-process
  - Many processes are running on a system: a high cost
- Inverted page tables has one entry for each frame in physical memory
  - Each entry consists of a pair <process-id, page-number>.
  - Only 1 page table in the system and it has only one entry for each page of physical memory

- For a memory reference: <process-id, page- number> *is searched in the page table*
  - If no match is found, then an illegal access
  - **Pros:** reduced memory needed to store each process's page table
  - **Cons:** need to search the whole page table for a page reference





Outline

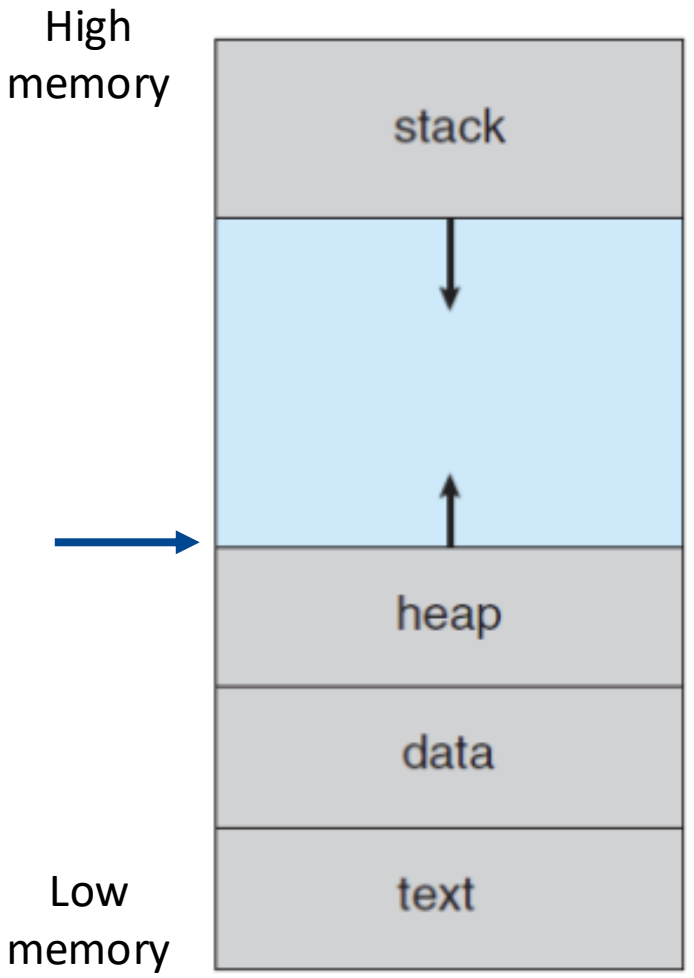
- Paging
- Page Tables:
  - Hierarchical Page Table
  - Hashed Page Table
  - Inverted Page Table
- **VM system calls**
- Linux's Buddy system and SLAB allocator

Recall a process's address space

- Dynamically allocated memory is in heap
  - **Breakpoint** marks the top of heap
- Addresses between breakpoint and stack all invalid

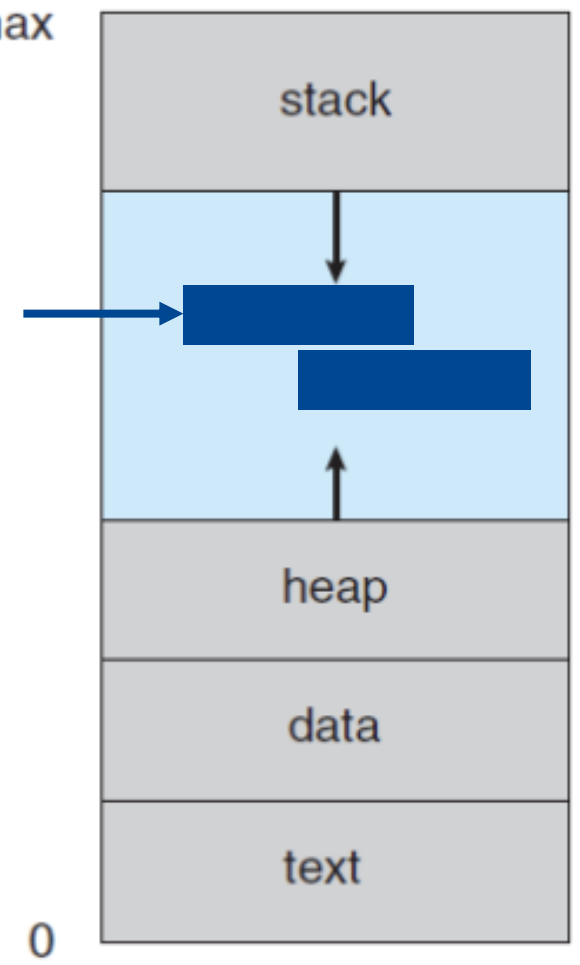
- OS tracks breakpoint.
- Accesses the memory regions between breakpoint and stack fault.

- Use syscall brk to set and return new value of breakpoint
  - char ***brk** (const char *addr);
- Use syscall sbrk to increment value of the breakpoint
  - char ***sbrk** (int incr);
- This is how *malloc* grows the heap



Memory Mapped Regions

- Memory mapped regions live between heap and stack
- Use the syscall **mmap** to map file specified by fd at virtual address addr
  - void *mmap (void *addr, size_t len, int prot, int flags, int fd, off_t offset)
- 2 types: file backed or anonymous (MAP_ANON)
- Private or Shared among processes
  - MAP_PRIVATE – modifications are private to a process (CoW)
  - MAP_SHARED – modifications visible by all processes (msync)
- Use huge pages: MAP_HUGETLB, MAP_HUGE_2MB, MAP_HUGE_1GB



MAP_PRIVATE and CoW

- the mapping's pages are initially shared read-only with the backing file
- the kernel only makes a process-private copy of a page when that process first writes to it

- Use Case in fork()
  - fork() logically copies the parent's entire address space. Copying gigabytes would be absurd.
  - Instead: copy only the page tables, and mark every writable page read-only and shared.
  - On the first write by either process, a **protection** fault fires; the **fault handler** copies just that one page and makes it writable.

More on Huge Pages

- Pros: reduce TLB misses and page walks. Good for throughput in big-footprint apps (databases, HPC, ML training). Cons?

- Two Ways to Get Huge Pages
  - Transparent Huge Pages (THP)
    - kernel automatically promotes / collapses
    - Controlled by madvise(MADV_HUGEPAGE); /sys/.../transparent_hugepage
    - Good for general apps, transparently
  - hugetlbfs (explicit)
    - Users with root reserve the size explicitly; a pre-set huge-page pool
    - Controlled by mmap(MAP_HUGETLB)
    - Good for databases / VMs needing guaranteed huge pages

More on Huge Pages

- There are limits of huge pages in a system

```
vboxuser@Debian13VM:~$ cat /proc/meminfo
MemTotal:        2012328 kB
MemFree:         1339388 kB
MemAvailable:    1785792 kB
Buffers:          118644 kB
Cached:           375580 kB
SwapCached:            0 kB
Active:           212400 kB
Inactive:         321840 kB
Active(anon):      40508 kB
```

```
FileHugePages:         0 kB
FilePmdMapped:         0 kB
CmaTotal:          65536 kB
CmaFree:           53504 kB
HugePages_Total:       0
HugePages_Free:        0
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:       2048 kB
Hugetlb:               0 kB
```

Problem in this example?

More VM System Calls

- int munmap(void *addr, size_t len)
  - Removes memory-mapped object, used in pair with mmap
- int msync(void *addr, size_t len, int flags);
  - Flush changes of mmapped file to backing store
- int mprotect(void *addr, size_t len, int prot)
  - Changes protection on pages to bitwise or of some PROT_. .. values
- int mincore(void *addr, size_t len, char *vec)
  - returns a vector that indicates whether pages of the calling process's virtual memory are resident in RAM



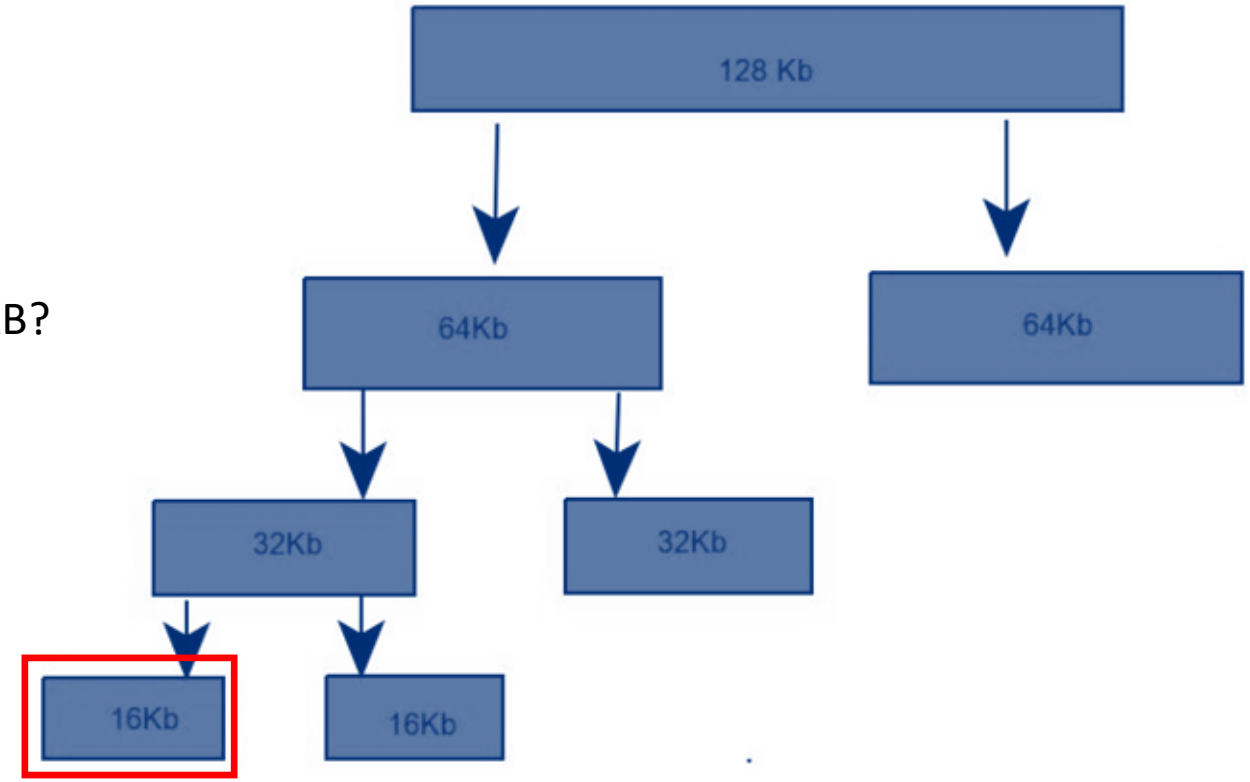
Outline

- Paging
- Page Tables:
  - Hierarchical Page Table
  - Hashed Page Table
  - Inverted Page Table
- VM system calls
- **Linux's Buddy system and SLAB allocator**

Buddy System

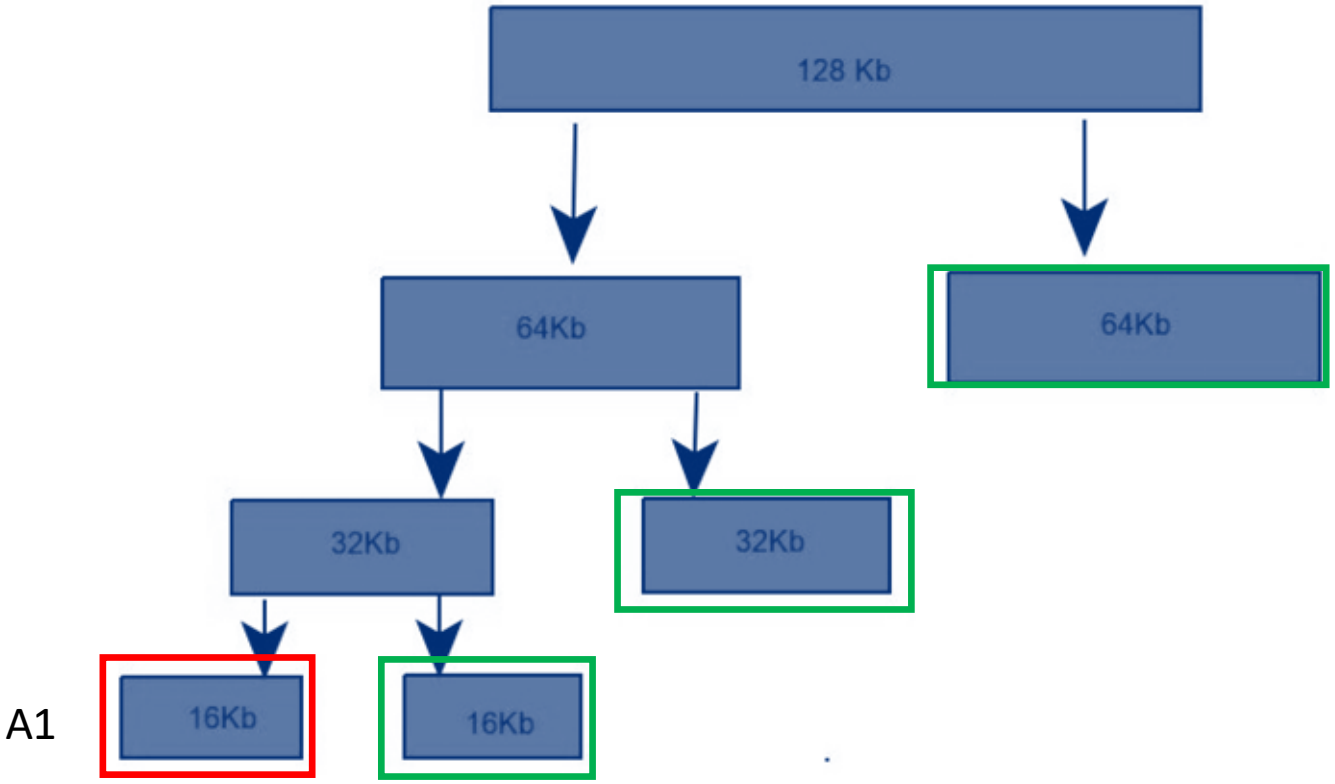
- The buddy system is the kernel's **physical frame allocator**
- Start from a large memory block, which will be divided into small parts to satisfy the request
- Memory Allocation uses a **power-of-2** allocator (4 KB, 8 KB, 16 KB …)
- When divided, the two smaller parts are of equal size and called as **buddies**
- One of the two buddies will be further divided into smaller parts until the request is fulfilled

A request of **13** KB?



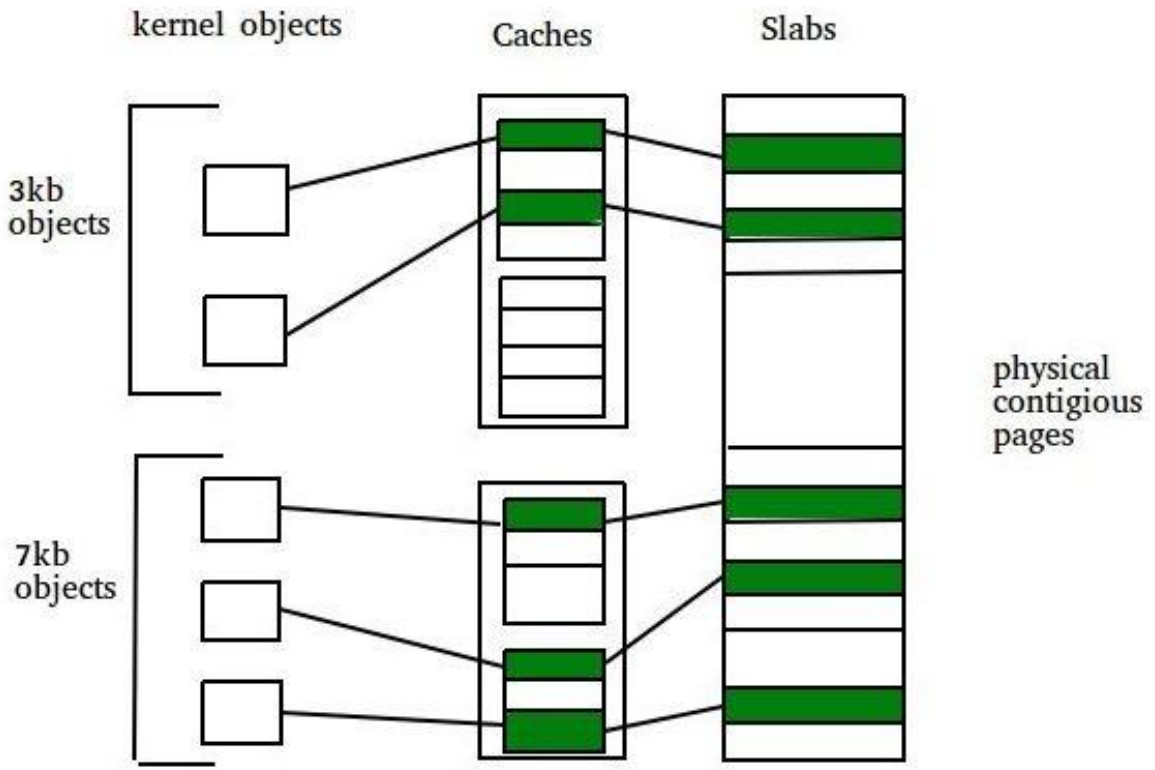
Buddy System

- Adjacent buddies can be combined to form a larger block: memory coalescing
- E.g., when block A1 that is allocated to the request of 13 KB previously is freed, it can be merged with its adjacent blocks



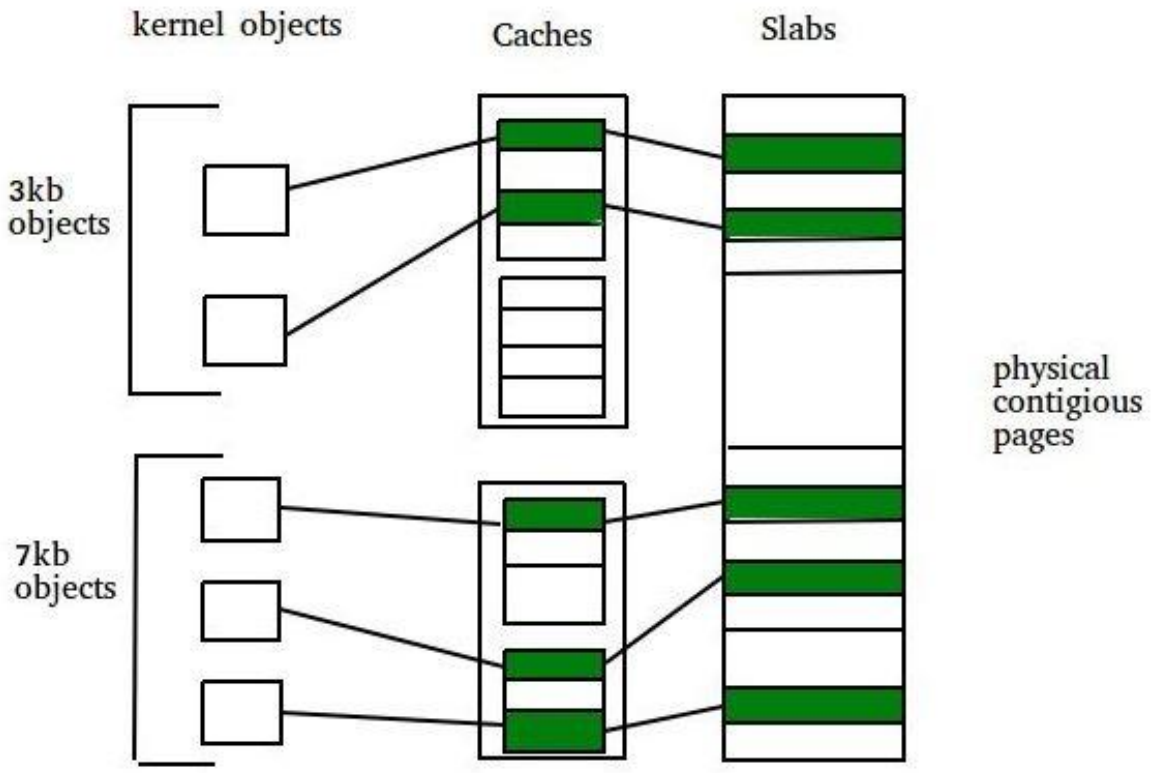
SLAB Allocator

- Buddy system still suffers from internal fragmentation: the smallest unit is a page size
- The kernel constantly allocates and frees many small, fixed-size objects: task_structs, inodes, file descriptors, etc
- **SLAB: built on buddy, carves pages into same-type objects and caches freed ones**.
- SLAB system removes internal fragmentation by reusing allocated memory for a data object of same type
  - E.g., Semaphore objects are the same type and size
  - Via slab caches



SLAB Allocator

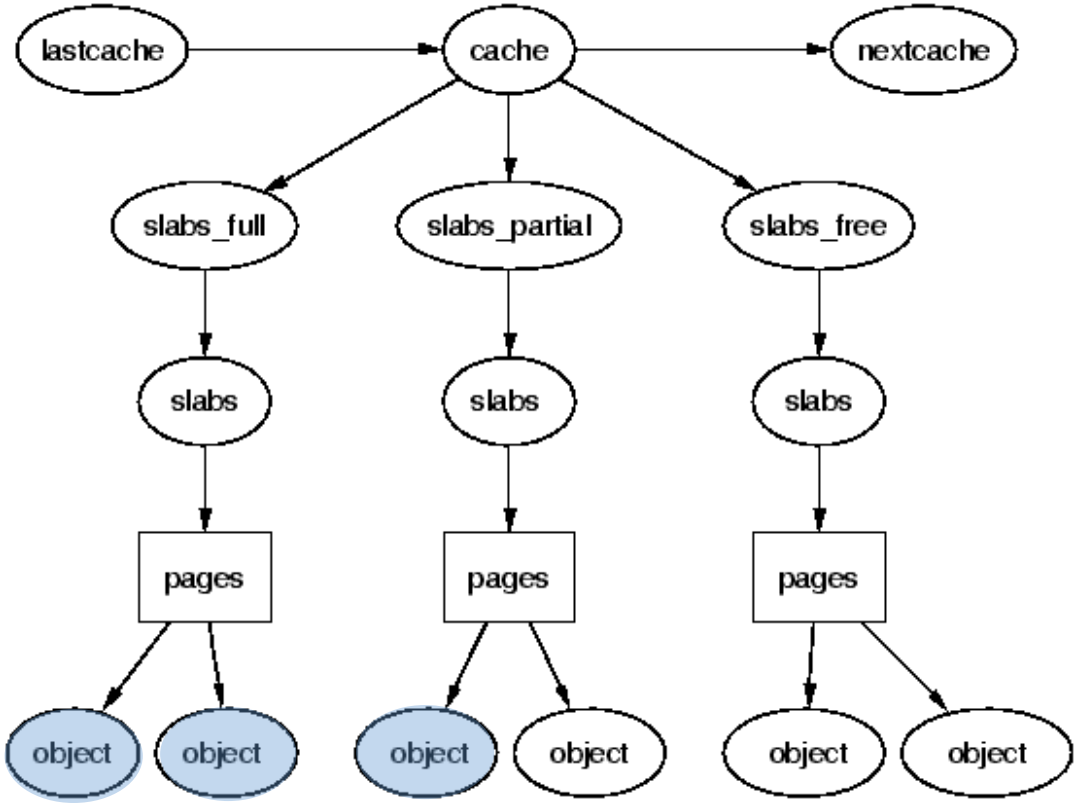
- For each unique kernel data structure (they need the same size allocation), there is a single **cache**
- E.g., a cache for the data structure representing process descriptors, a cache for file objects, a cache for inodes (metadata for filesystem), etc.



SLAB Allocator

- A slab might be **Full**, **Empty**, or **Partial.** E.g., a 12KB slab can store four 3KB objects.
- Give a request, the slab allocator tries to find a free object in a Partial slab
- If no partial slab exists, an object from an Empty slab will be used
- If no Empty slab exists, a new Slab will be allocated and assign to a cache

A request of **13** KB?



User-Space Allocators

- The kernel hands a process big chunks via brk/mmap.
- ***malloc*** carves them into small objects : the user-space analogue of buddy + SLAB
- Two sources:
  - Small requests: from the heap, grown with brk/sbrk.
  - Large requests (> ~128 KB, mmap): a fresh `mmap`, returned to the OS on free.

- glibc's allocator (**ptmalloc**) keeps free blocks in **bins** by size: fast bins, small bins, large bins
- Multiple **arenas** (per-thread heaps) cut lock contention in multithreaded programs.

Allocator	Origin	Strength
ptmalloc	glibc (default)	general-purpose
tcmalloc	Google	fast; per-thread caches; low **fragmentation**
jemalloc	FreeBSD / Meta	**fragmentation** control; profiling; arenas
mimalloc	Microsoft	small, fast, good locality

Back to the Questions

- Q1 How does paging translate a logical address to physical?
  - Split the address: calculate offset d, page no. p. Look up p → frame f. Physical = $f \cdot 2^n + d$.
- Q2 Why a TLB, and how do you compute effective access time?
  - Single level, multi-level TLB
- Q3 Why hierarchical, hashed, and inverted page tables?
  - A flat 64-bit table is impossible. Hierarchical pages the table only touched sub-tables allocated. Hashed suits sparse/large spaces. Inverted keeps one table.